

Aulão OBI - 2026

Primeira fase

Fernando Kiotheka Roberto Tomchak Pedro Vinicius
Antonio Filho Nathan Endo Matheus Telles

UFPR

30/05/2026



Programação Competitiva

- Quando se fala em Maratona, se fala em **programação competitiva**
- Esporte mental (como xadrez e *e-sports*)
- Participantes criam um programa que resolva um problema computacional conforme uma especificação
- Conteúdo da especificação: informação que o programa recebe e o que é uma resposta correta
- Programa deve ser eficiente
- Diversas linguagens de programação são aceitas
- Todos os participantes recebem os mesmos problemas - vence quem resolver mais e mais rápido

Competições

- **Maratona de Programação (da SBC):** estudantes do ensino superior (times de três)
- **Olimpíada Brasileira de Informática:** estudantes do ensino fundamental até o primeiro ano de graduação (individual)
- **Outras competições:** Meta Hacker Cup, TopCoder, Codeforces, etc. (geralmente individuais)

Motivação (1)

Resolução de problemas Foco e capacidade de resolver problemas são melhorados porque a programação competitiva exercita essas bases constantemente.

Trabalho em equipe A prova de maratona é tensa e precisa de muita organização por parte do time. Esse ambiente proporciona a construção de uma boa dinâmica de equipe e desenvolvimento de habilidades de trabalho em equipe úteis também fora da maratona.

Motivação (2)

Camisetas e balões As camisetas são confortáveis e os balões são agradáveis!



Motivação (3)

Conceitos de computação Maratona usa desde cedo conceitos que só aparecem mais tarde na graduação, facilitando as matérias. Isso sem contar assuntos que não são abordados em sala!

Mercado de trabalho Grandes empresas gostam de bons competidores e vão às competições buscá-los!

Motivação (4)

Viagens As provas nacionais e mundiais acontecem sempre em lugares diferentes. É uma oportunidade de conhecer esses lugares!



UFPR na Maratona

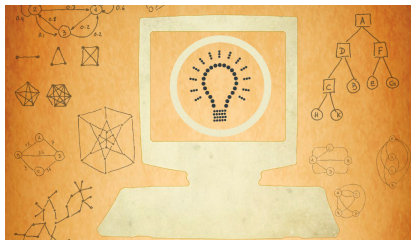
A UFPR já foi a 15 finais nacionais e 3 finais mundiais.

- Florianópolis
- Belo Horizonte
- Vila Velha
- Estocolmo, Suécia
- Campinas
- Joinville
- Orlando, EUA
- Goiânia
- Varsóvia, Polônia
- Londrina
- Ponta Grossa
- Uberlândia
- Fortaleza
- Foz do Iguaçu
- Campina Grande
- Gramado
- Campo Grande
- Chapecó

UFPR na Maratona



Olimpíada Brasileira de Informática



- Organizada pela UNICAMP para despertar o interesse no estudo da computação.
- Ensino fundamental e no ensino médio, mas também no ensino superior.
- A modalidade que você calouro poderá participar é de nível Sênior na Modalidade Programação.
- **Apenas no primeiro ano de graduação**
- A prova é feita individualmente, sem consulta a materiais externos.

Maratona da SBC



- A “maratona de verdade”
- Quatro fases: regional, nacional, latino-americana e mundial
- 5 horas de prova, cerca de 13 problemas
- Time recebe um balão sempre que resolve um problema
- **Elegível se:** completa até 23 anos no ano atual **ou** começou a primeira graduação até 4 anos atrás

Formato da Maratona

- Participantes podem levar coisas impressas (livro, apostila, caderno)
- Nenhum acesso à internet
- Você submete seu código e ele é executado com vários casos de teste (verificando se sua solução funciona)

Julgamento e resultado

Após o juiz (virtual) testar seu código, você recebe um veredito:

AC – Accepted Seu programa passa em todos os casos de teste;

WA – Wrong Answer Seu programa entregou uma resposta incorreta em pelo menos um dos casos de teste;

TLE – Time Limit Exceeded Seu programa demorou para responder um caso de teste dentro do limite de tempo que o problema aceita;

MLE – Memory Limit Exceeded Seu programa excedeu o limite de memória em algum caso de teste;

CE – Compilation Error Seu programa apresentou erro na compilação;

RE – Runtime Error Seu programa falhou durante a execução. Isso significa um acesso de memória inválida, uma divisão por zero, etc.

Exemplo de problema - descrição

Problema do aquecimento da 1.^a fase da Maratona SBC 2018

Problema A Jogo de boca

Um jogo infantil, muito popular, é o *21 de boca*. O jogo é jogado da seguinte forma: o primeiro jogador diz um número, n_0 , que pode ser 1 ou 2. O segundo jogador pode então dizer um número n_1 tal que $n_1 \in \{n_0 + 1, n_0 + 2\}$. E assim por diante, os jogadores se alternam, dizendo sempre um número que é um ou dois maior do que o anterior. O jogador que disser 21 ganha o jogo. Por exemplo, a sequência de números poderia ser: 1, 3, 5, 6, 7, 9, 11, 12, 14, 15, 16, 18, 19, 21. Neste jogo, o primeiro jogador sempre perde, se o segundo souber jogar bem.

A cada nova geração as crianças ficam mais espertas. Atualmente, apesar de acharem o *21 de boca* um jogo interessante, muitas crianças não se sentem desafiadas o bastante e por isso resolveram generalizar o jogo, criando assim o *N de boca*. Dado um inteiro N , no lugar do 21, o primeiro jogador pode escolher 1 ou 2. A partir daí os jogadores se alternam, adicionando 1 ou 2 ao número anterior, até que um deles diga o número N e ganhe o jogo. Sabendo que ambos os jogadores são excelentes e sabem jogar muito bem, seu problema é determinar qual o inteiro inicial que o primeiro jogador deve escolher para ganhar o jogo.

Exemplo de problema - entrada e saída

Entrada

A entrada consiste de uma única linha que contém o inteiro N ($3 \leq N \leq 10^{100}$) escolhido para a partida atual do *N de boca*.

Saída

Seu programa deve produzir uma única linha com um inteiro representando o número, em $\{1, 2\}$, que o primeiro jogador deve escolher, para ganhar o jogo. Se não for possível, então o inteiro deve ser zero.

Exemplo de entrada 1 7	Exemplo de saída 1 1
Exemplo de entrada 2 9	Exemplo de saída 2 0
Exemplo de entrada 3 12341234123412341234123412341234	Exemplo de saída 3 2

Como se preparar

- Praticar problemas
- Ensinar colegas
- Estudar (livros, aulas, etc.)
- Refletir sobre os problemas (as soluções chegam!)

Clube de Programação UTFPR-CT

- Treinos
- Discussão em grupo
- Apoio mútuo
- Reuniões semanais com o professor Leandro Zatesko
- `cdp.dainf.ct.utfpr.edu.br`



Capimara UFPR

- Discussão em grupo
- Apoio mútuo
- **CI1031 - Desafios de Programação:** disciplina optativa de maratona
- Treinos, Internas, ...



Mais informações: inf.ufpr.br/maratona

- Livreto de introdução à Maratona
- Material de apoio
- Grupo no Telegram

Venha falar com a gente!

Do Pascal ao C++

Operações em C++

Básicas

+	soma
-	subtração
*	multiplicação
/	divisão
%	módulo

Bitwise

&	AND
	OR
^	XOR
~	NOT
<<	shift esq.
>>	shift dir.

Min / Max

min(a, b)
max(a, b)
min({a,b,c})
max({a,b,c})

Operadores de Condição e Atribuição em C++

Condição / Comparação

==	igual
!=	diferente
>	maior
<	menor
>=	maior ou igual
<=	menor ou igual
&&	E lógico (AND)
	OU lógico (OR)
!	NÃO lógico (NOT)

Atribuição

=	atribuição simples
+=	soma e atribui
-=	subtrai e atribui
*=	multiplica e atribui
/=	divide e atribui
%=	módulo e atribui
++	incremento
--	decremento

Tipos de Variáveis

Tipo	Pascal	C++	Observação
Inteiro	Integer	int	-2^{31} a $2^{31} - 1$
Inteiro longo	Int64	long long	-2^{63} a $2^{63} - 1$
Real	Single	float	~7 dígitos
Real duplo	Double	double	~15 dígitos
Caractere	Char	char	Aspas simples
Booleano	Boolean	bool	True vs true
String	String	string	
Vetor	array[1..n]	vector<T>	1 vs 0-based

Inicialização de Variáveis — Sintaxe

Pascal

```
var
  n: Integer;
  x: Double;
  c: Char;
  s: String;
  b: Boolean;
```

```
begin
  n := 10;
  x := 3.14;
  c := 'A';
  s := 'ola';
  b := True;
end.
```

C++

```
int n = 10;
double x = 3.14;
char c = 'A';
string s = "ola";
bool b = true;
```

// ou sem valor inicial

```
int n;
double x;
```

Em Pascal, declaração e atribuição são separadas. Em C++, podem ser feitas juntas.

Escopo — Global vs Local

Pascal

```
var
  global: Integer; // global

procedure exemplo;
var
  local: Integer; // local
begin
  local := 5;
end;

begin
  global := 10;
end.
```

C++

```
int global; // global

void exemplo() {
  int local = 5; // local
}

int main() {
  global = 10;
}
```

Variáveis globais em C++ são iniciadas com 0 automaticamente. Variáveis locais **não** são iniciadas — podem conter lixo de memória.

Vector em C++

Declaração e inicialização

```
vector<int> v;  
vector<int> v(5, 0);  
vector<int> v = {1,2,3};
```

Acesso e Informações

```
v[i];  
v.at(i);  
v.front();  
v.back();
```

Inserção e remoção

```
v.push_back(x);  
v.pop_back();  
v.insert(v.begin()+i, x);  
v.erase(v.begin()+i);
```

```
v.size();  
v.empty();  
v.clear();  
v.resize(n);
```

Vector em C++ (Continuação)

Ordenar

```
sort(v.begin(), v.end());
```

Inverter

```
reverse(v.begin(), v.end());
```

Contar ocorrências de x

```
count(v.begin(), v.end(), x);
```

Loops de repetição

```
// indice
for(int i = 0; i < v.size(); i++){
    v[i] = i+1;
}
```

```
// range-based (moderno)
for(int x : v){
    x += 5;
}
```

```
for(auto &x : v) { cin >> x; }
```

```
while (condição) {
    //Lógica..
}
```

Pair

Declaração e inicialização

```
pair<int, int> p;  
p = {1, 2};  
p = make_pair(1, 2);
```

Acesso

```
p.first;  
p.second;
```

Uso com vector

```
vector<pair<int, int>> v;  
v.push_back({1, 2});  
v.push_back({3, 4});
```

```
// ordenar por first  
// desempata pelo second  
sort(v.begin(), v.end());
```

Comparação

```
// compara first,  
//depois second  
if(p1 < p2) { ... }
```

Set

Declaração e inserção

```
set<int> s;  
s.insert(x); //  $O(\log n)$   
s.erase(x); //  $O(\log n)$   
s.clear();
```

Consulta

```
s.count(x); // 0 ou 1  
s.size();  
s.empty();
```

Propriedades

```
// sem duplicatas  
// sempre ordenado  
// acesso  $O(\log n)$ 
```

Iteração

```
for(int x : s) { ... }  
  
// menor elemento  
*s.begin();  
// maior elemento  
*s.rbegin();
```

Por que C++?

- C++ é C com baterias incluídas, com muitas estruturas de dados e algoritmos prontos
- A biblioteca padrão, a STL já contém implementações de:
 - Alocação dinâmica (vetores que crescem de tamanho sozinhos)
 - Árvores balanceadas (geralmente rubro-negras)
 - Filas de prioridade
 - Ordenação
 - Pares (ordenação já embutida)
 - Manipulação de strings de forma dinâmica
 - Entrada/Saída mais concisa

Estrutura básica de um código em C++

Aviso que não são “boas práticas de programação”. Usar isso em código *enterprise* é pedir pra ser demitido.

1. Incluímos todos os cabeçalhos da STL de uma vez só (equivalente a `#include <set>`, `#include <vector>`, etc).

Exclusivo ao GCC e aumenta o tempo de compilação.

```
#include <bits/stdc++.h>
```

2. Usamos o espaço de nomes `std`. Assim, pra usar um `vector` não precisamos digitar `std::vector`.

```
using namespace std;
```

3. Função `main` padrão.

```
int main() {
```

4. Implemente sua lógica.

Estrutura básica de um código em C++ (continuado)

```
#include <bits/stdc++.h>
using namespace std;

int main() {

    // seu código aqui

}
```

Compilando C++

Podemos usar tanto o GCC (g++) ou usando o Make:

```
g++ programa.cpp -O2 -o programa && ./programa
```

É recomendado compilar com -O2 para ter resultados mais próximos ao do juiz.

```
make programa
```

Assume que existe um código chamado programa.cpp, após executar esse comando será gerado um executável com o nome programa.

Saída em C++

Para escrevermos na saída padrão, usamos o cout com <<. **Use** "\n" para quebra de linha

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "Ola mundo!\n";
}
```

Entrada	Saída
	Ola mundo!

Entrada em C++

Para ler da entrada padrão, usamos o `cin` com `>>`. O **tipo da variável** muda o jeito como é lido.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int x; char c; string s;
    cin >> x >> c;
    getline(cin, s); // le a linha inteira
    cout << "x=" << x << " c=" << c << "\n";
    cout << s << "\n";
}
```

Entrada	Saída
1 a Hello World!	x=1 c=a Hello World!

Lendo até EOF

Muitas vezes é necessário ler até o fim do arquivo. O objeto `cin` pode ser convertido em booleano para indicar se a entrada ainda é válida.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int x;
    while (cin >> x) {
        cout << x << ".";
    }
    cout << "\n";
}
```

Entrada	Saída
1 2 3 4 5 6 7 8	1.2.3.4.5.6.7.8.